

Project Documentation: Dual-Edge Moore Detector Unit

Digital Design Lab

Project Overview

This lab implements a **Moore Finite State Machine (FSM)** for dual-edge (any-edge) detection in Verilog HDL[cite: 32]. The circuit monitors an input signal (`level`) and generates a single clock-cycle pulse (`tick`) whenever either a rising edge ($0 \rightarrow 1$) or a falling edge ($1 \rightarrow 0$) is detected[cite: 32]. As a Moore FSM, the output `tick` depends strictly on the state machine being in the intermediate `EDGE` state[cite: 32].

Specifications & Functionality

- **States:**
 - `ZERO` (`2'b00`): Stable low input state[cite: 32]. Output `tick` = 0[cite: 32]. Transitions to `EDGE` on rising edge (`level` = 1)[cite: 32].
 - `EDGE` (`2'b01`): Edge detection state[cite: 32]. Output `tick` = 1[cite: 32]. Transitions to `ONE` if `level` = 1, or returns to `ZERO` if `level` = 0[cite: 32].
 - `ONE` (`2'b10`): Stable high input state[cite: 32]. Output `tick` = 0[cite: 32]. Transitions to `EDGE` on falling edge (`level` = 0)[cite: 32].
- **Asynchronous Reset:** Active-high reset forces state register to `ZERO` on posedge `reset`[cite: 32].
- **Moore Output Logic:** Asserted high (`tick` = 1) if and only if `current_state` == `EDGE`[cite: 32].

Port Summary

Port Name	Direction	Type	Width	Description
<code>clk</code>	Input	wire	1	Clock input signal[cite: 32, 33]
<code>reset</code>	Input	wire	1	Asynchronous active-high reset[cite: 32, 33]
<code>level</code>	Input	wire	1	Input level signal to monitor[cite: 32, 33]
<code>tick</code>	Output	reg	1	Pulse output on any level change[cite: 32, 33]

Table 1: Dual-Edge Moore Detector Port Interfaces

Verilog Source Code

Design Module (`edge_moore.v`)

```
module edge_moore (
    input wire clk,
    input wire reset,
    input wire level,
    output reg tick
);

parameter ZERO = 2'b00;
parameter EDGE = 2'b01;
parameter ONE  = 2'b10;

reg [1:0] current_state, next_state;

// State Register with Asynchronous Reset
always @(posedge clk or posedge reset) begin
    if (reset)
        current_state <= ZERO;
    else
        current_state <= next_state;
end

// Next State Logic
always @(*) begin
    case (current_state)
        ZERO: begin
            if (level)
                next_state = EDGE; // 0 -> 1 transition
            else
                next_state = ZERO;
        end
        EDGE: begin
            if (level)
                next_state = ONE;
            else
                next_state = ZERO;
        end
        ONE: begin
            if (!level)
                next_state = EDGE; // 1 -> 0 transition
            else
                next_state = ONE;
        end
        default:
            next_state = ZERO;
    endcase
end

// Moore Output Logic
always @(*) begin
    if (current_state == EDGE)
        tick = 1'b1;
    else
        tick = 1'b0;
end
```

```
endmodule
```

Testbench Module (tb_edge_moore.v)

```
'timescale 1ns / 1ps

module tb_edge_moore;

    reg clk;
    reg reset;
    reg level;

    wire tick;

    edge_moore DUT (
        .clk      (clk),
        .reset    (reset),
        .level    (level),
        .tick     (tick)
    );

    // Clock Generation (10ns Period)
    initial begin
        clk = 1'b0;
        forever #5 clk = ~clk;
    end

    // Test Stimulus
    initial begin
        // Initial values
        reset = 1'b1;
        level = 1'b0;
        #10;

        reset = 1'b0;
        #10;

        level = 1'b1;
        #40;

        level = 1'b0;
        #40;

        level = 1'b1;
        #20;

        level = 1'b0;
        #20;

        $finish;
    end

    // Output Display Monitor
    initial begin
        $monitor(
```

```
        "Time = %0t | clk = %b | reset = %b | level = %b | tick = %  
          b",  
        $time,  
        clk,  
        reset,  
        level,  
        tick  
    );  
end  
  
endmodule
```

Simulation & Verification

Execution Command

To execute batch simulation using Cadence Xcelium or ModelSim:

```
xrun edge_moore.v tb_edge_moore.v
```

Expected Simulation Output

The simulation log confirms that `tick` pulses for one clock period following both rising ($0 \rightarrow 1$) and falling ($1 \rightarrow 0$) transitions on `level`[cite: 32, 33]:

```
Time = 0 | clk = 0 | reset = 1 | level = 0 | tick = 0  
Time = 5 | clk = 1 | reset = 1 | level = 0 | tick = 0  
Time = 10 | clk = 0 | reset = 0 | level = 0 | tick = 0  
Time = 20 | clk = 0 | reset = 0 | level = 1 | tick = 0  
Time = 25 | clk = 1 | reset = 0 | level = 1 | tick = 1  
Time = 35 | clk = 1 | reset = 0 | level = 1 | tick = 0  
Time = 60 | clk = 0 | reset = 0 | level = 0 | tick = 0  
Time = 65 | clk = 1 | reset = 0 | level = 0 | tick = 1  
Time = 75 | clk = 1 | reset = 0 | level = 0 | tick = 0  
Time = 100 | clk = 0 | reset = 0 | level = 1 | tick = 0  
Time = 105 | clk = 1 | reset = 0 | level = 1 | tick = 1
```